ELO320 - Estructuras de Datos y Algoritmos
Introducción a C
TDA y Listas Enlazadas

Dr. Nicolás Gálvez R.
nicolas.galvez@usm.cl

Ing. Civil Telemática - Departamento de Electrónica

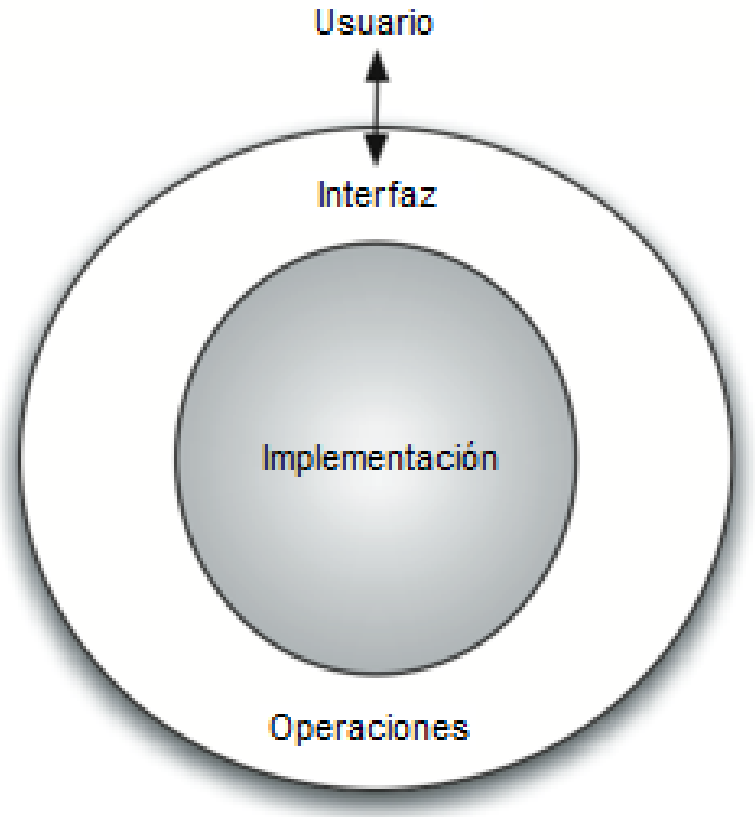
2do Semestre 2022

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

TDA: Tipos de Datos Abstracto

- **Tipo de Datos**: Colección de valores y las operaciones que se pueden realizar con éstos. Por ejemplo, enteros (int) con operadores aritméticos.
- **Tipo de Datos Abstracto**: Es una materialización de un tipo de datos como componente de software. Se caracteriza por:
  - Interfaz: conjunto de operaciones.
  - Entradas y salidas interactuan con la interfaz.
  - Implementación oculta.
  - **La única manera de acceder y manipular los datos del TDA es mediante las operaciones de la interfaz**.

TDA: Tipos de Datos Abstracto



TDA: Tipos de Datos Abstracto

x=a+b;

x=suma(a,b);

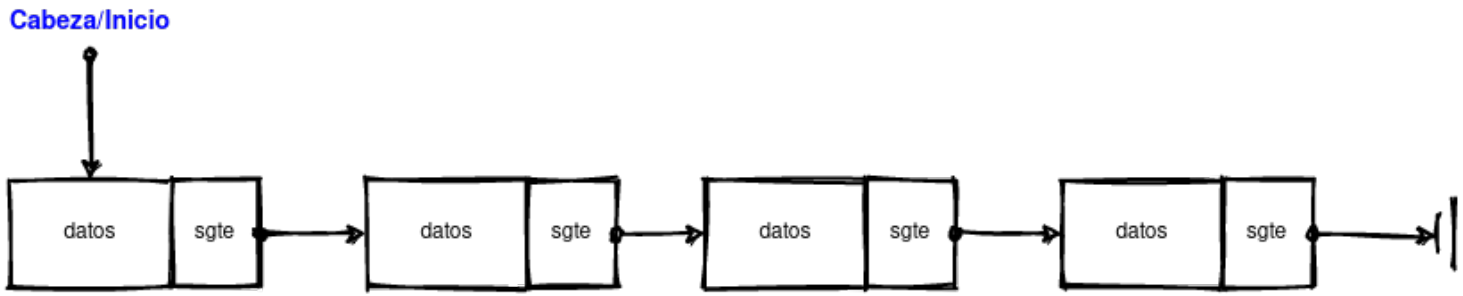
Problemas con Arreglos

- Los arreglos estáticos ofrecen acceso eficiente a la información.
- Los arreglos son muy útiles en casos de memoria limitada.
- Pero, ¿qué pasa con arreglos de forma dinámica?
  - ¿Qué pasa si elimino un elemento del arreglo? Debo correr los elementos separados.
  - ¿Qué pasa si ingreso un nuevo elemento? Debo correr todos los elementos posteriores.
  - En el peor de los casos, se deben mover todos los elementos una vez. Por lo tanto serán N instrucciones.
  - Poco eficiente. $O(n)$
- Las listas enlazadas ayudan a manejar esto eficientemente: Simple, Doble y Circular.

Lista Simple Enlazada

Una lista simple enlazada es un TDA que genera un enlace entre nodos a través de punteros. Ventajas:

- No tienen que estar ordenadas correlativamente en memoria.
- No es necesario desplazar los elementos en inserciones o eliminaciones. Se modifican los punteros.



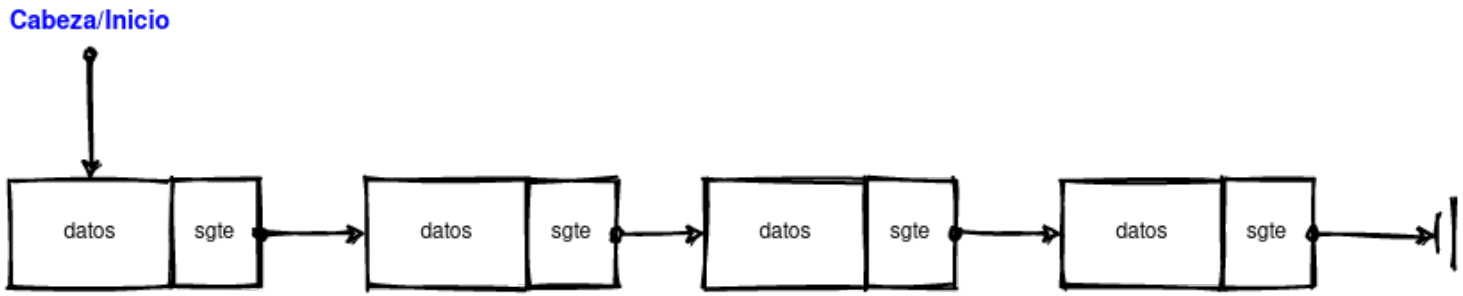
Está compuesta por:

- Nodos: Contienen la información en lugares de memoria no ordenados.
- Punteros: Enlazan los distintos nodos para conformar la lista.

Lista Simple Enlazada

Los punteros permiten al menos:

- Controlar el inicio de la lista.
- Apuntar al siguiente elemento en la lista.
- Acceder a cada nodo.

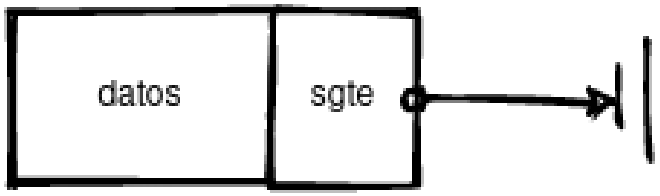


Lista Simple Enlazada: Nodos

Los nodos son definidos con un struct, que almenos contiene un tipo de dato y un puntero.

C

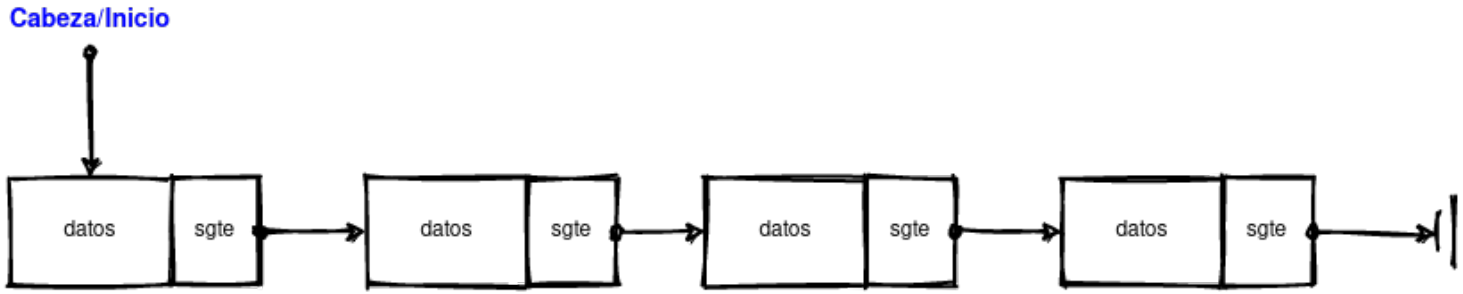
```
struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo, enlace
};

typedef struct le_nodo{
        float datos[20]; //dato, puede ser lo que uds. necesiten
        struct le_nodo *sgte; // puntero a un nodo, enalce.
} nodo_t;
```



Lista Simple Enlazada: Interfaz

Al igual que Python (`list()` o `[]`), las listas tienen un conjunto de funciones que interactúan como interfaz:

- Inicializar Lista.
- Agregar un elemento a una lista.
- Determinar el largo de una lista.
- Eliminar un elemento de una lista.
- Imprimir una lista.
- Limpiar una lista.



Lista Simple Enlazada: Inicializar

Las listas, en su versión más básica, dependen de un puntero al inicio de la lista. Éste suele llamarse *header*, *cabecera* o *inicio*.

C

```c
#include<stdio.h>

typedef struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo enalce.
} nodo_t;

int main(int argc, char **argv){

        nodo_t * inicio = NULL;
        // Inicializamos la lista, con su puntero de inicio apuntando a NULL
        // Equivalente a lista vacía []

        return 0;
}
```

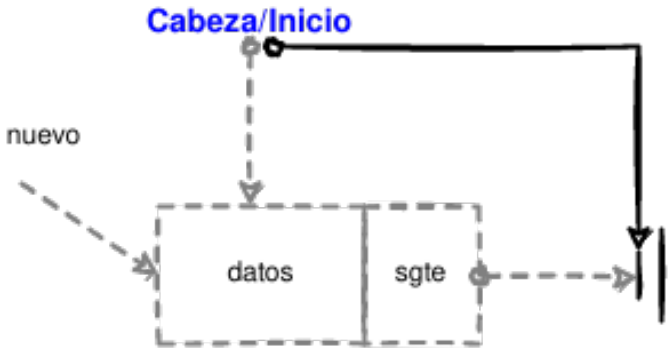
Lista Simple Enlazada: Añadir nodos

Tendremos 4 casos a tener en consideración.

- Añadir primer nodo $\rightarrow$ lista vacía.
- Añadir al inicio.
- Añadir entre nodos.
- Añadir al final.

Lista Simple Enlazada: Primer nodo

Esquema:

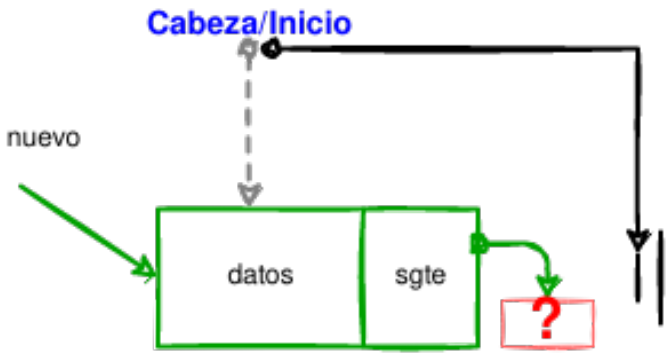


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Puntero siguiente de nuevo (`nuevo->sgte`) apunta a `NULL`.
4. Puntero `inicio` apunta a nuevo.

Lista Simple Enlazada: Primer nodo

Esquema:

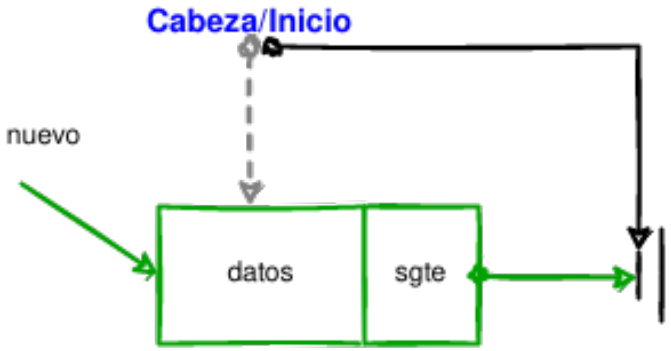


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Puntero siguiente de nuevo (`nuevo->sgte`) apunta a `NULL`.
4. Puntero `inicio` apunta a nuevo.

Lista Simple Enlazada: Primer nodo

Esquema:

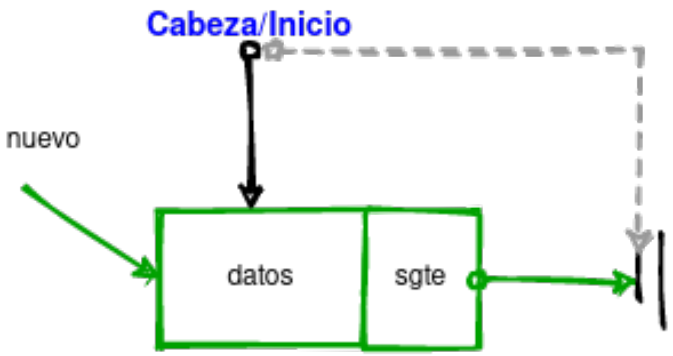


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Puntero siguiente de nuevo (`nuevo->sgte`) apunta a `NULL`.
4. Puntero `inicio` apunta a nuevo.

Lista Simple Enlazada: Primer nodo

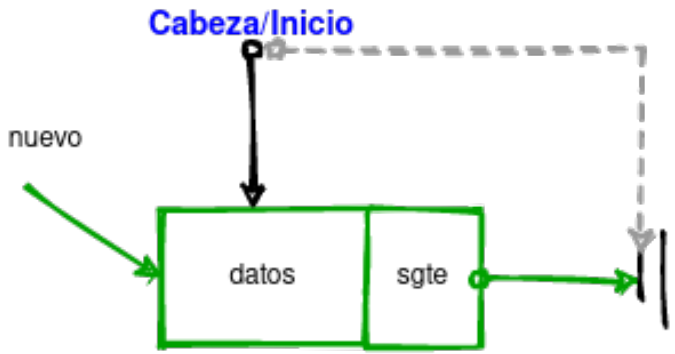
Esquema:



Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Puntero siguiente de nuevo (`nuevo->sgte`) apunta a `NULL`.
4. Puntero `inicio` apunta a nuevo.

Lista Simple Enlazada: Primer nodo

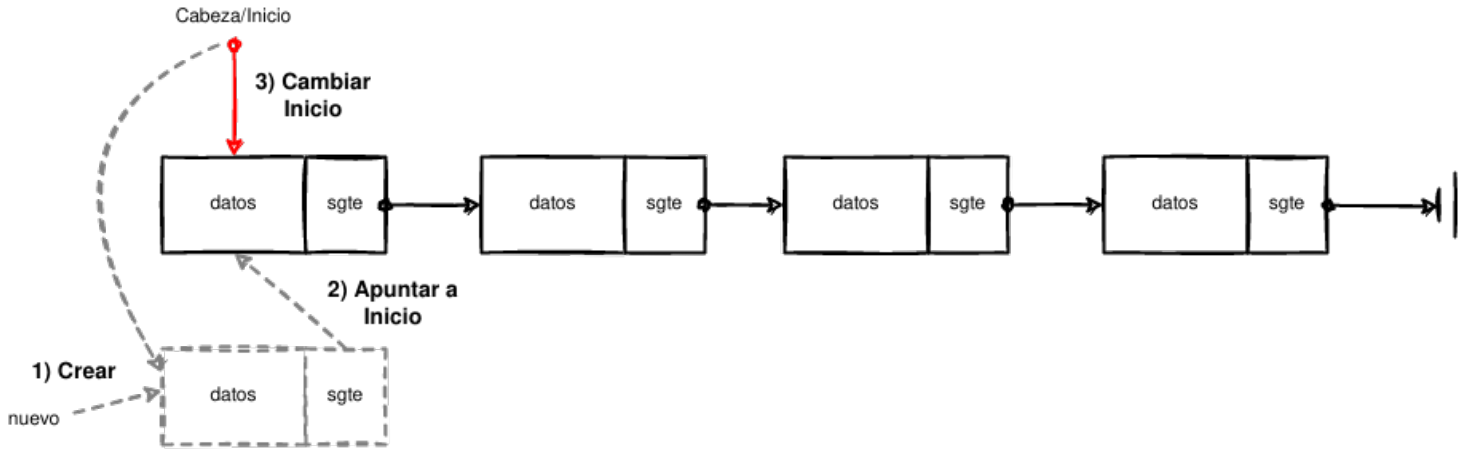


C

```
/* No complicarse: ** = puntero pasado por referencia
para que dirección de memoria cambie afuera de la función */
void first_node(nodo_t **cabeza, int dato){

        nodo_t *nuevo = malloc(sizeof(nodo_t)); // 1) crea nodo
        nuevo->dato = dato; // 2) asigna dato
        nuevo->sgte = NULL; // 3) siguiente apunta a NULL
        *cabeza = nuevo; // 4) inicio apunta a nuevo
}
```

Lista Simple Enlazada: Añadir al inicio

Esquema:

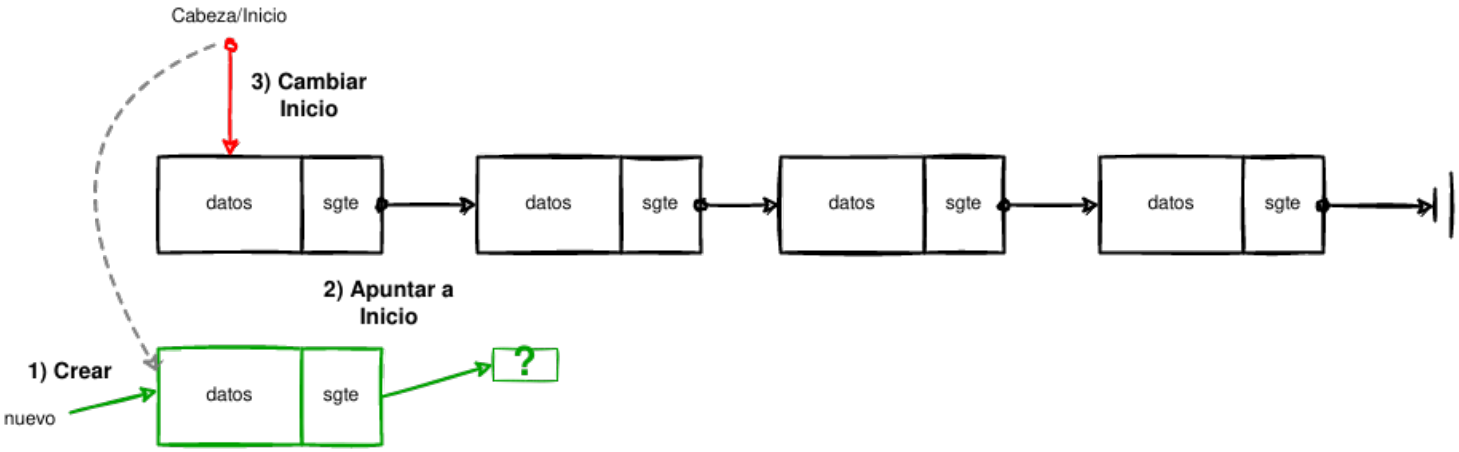


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Puntero siguiente de nuevo (`nuevo->sgte`) apunta a `inicio`.
4. Puntero `inicio` apunta a nuevo.

Lista Simple Enlazada: Añadir al inicio

Esquema:

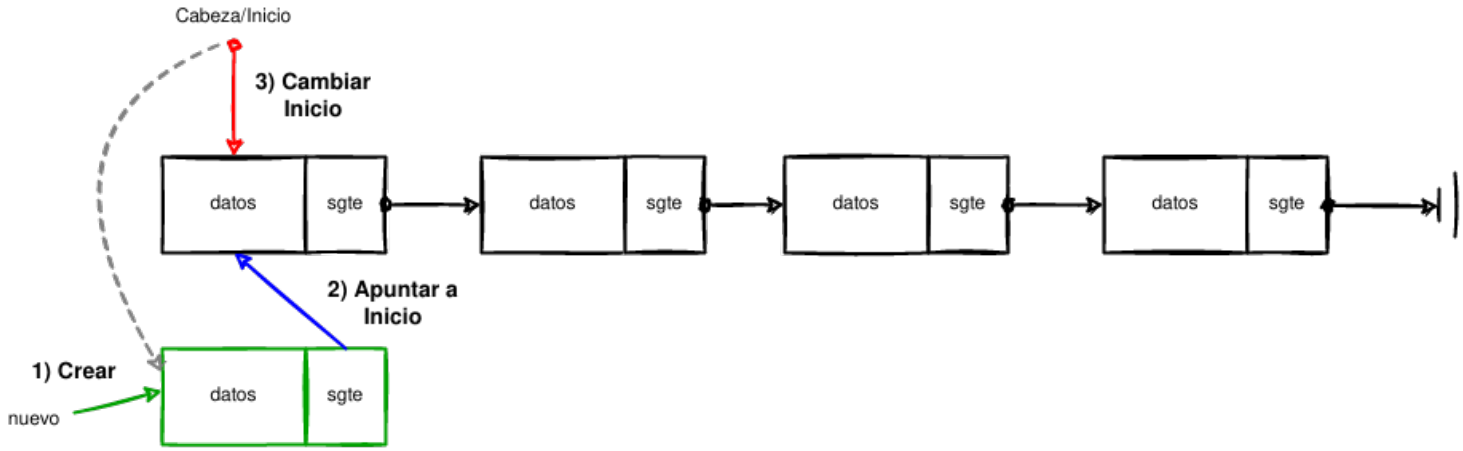


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Puntero siguiente de nuevo (`nuevo->sgte`) apunta a `inicio`.
4. Puntero `inicio` apunta a nuevo.

Lista Simple Enlazada: Añadir al inicio

Esquema:

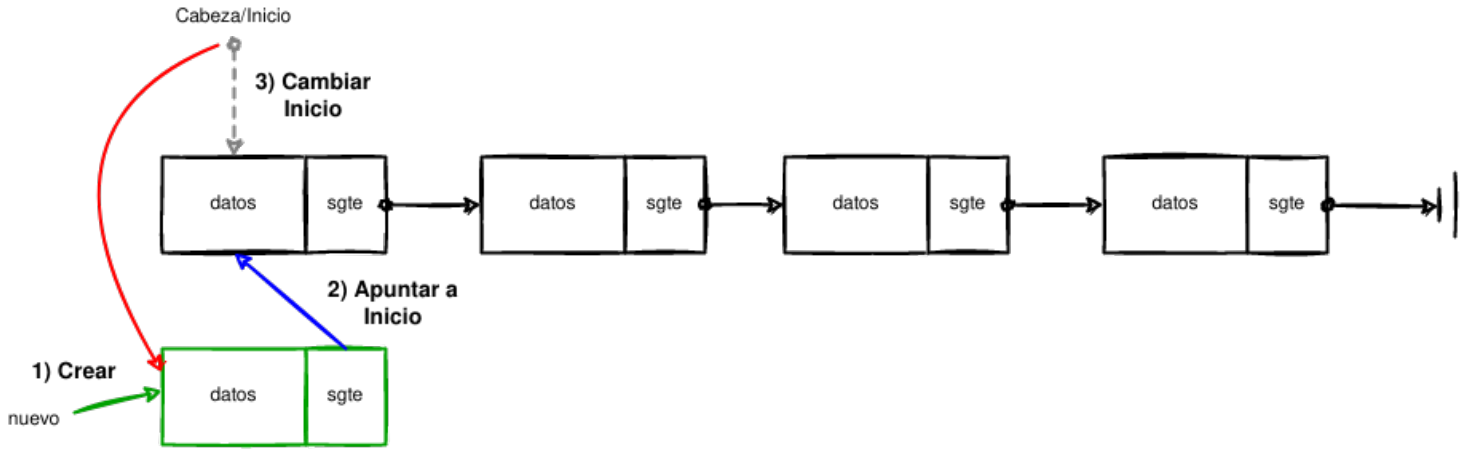


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Puntero siguiente de nuevo (`nuevo->sgte`) apunta a `inicio`.
4. Puntero `inicio` apunta a nuevo.

Lista Simple Enlazada: Añadir al inicio

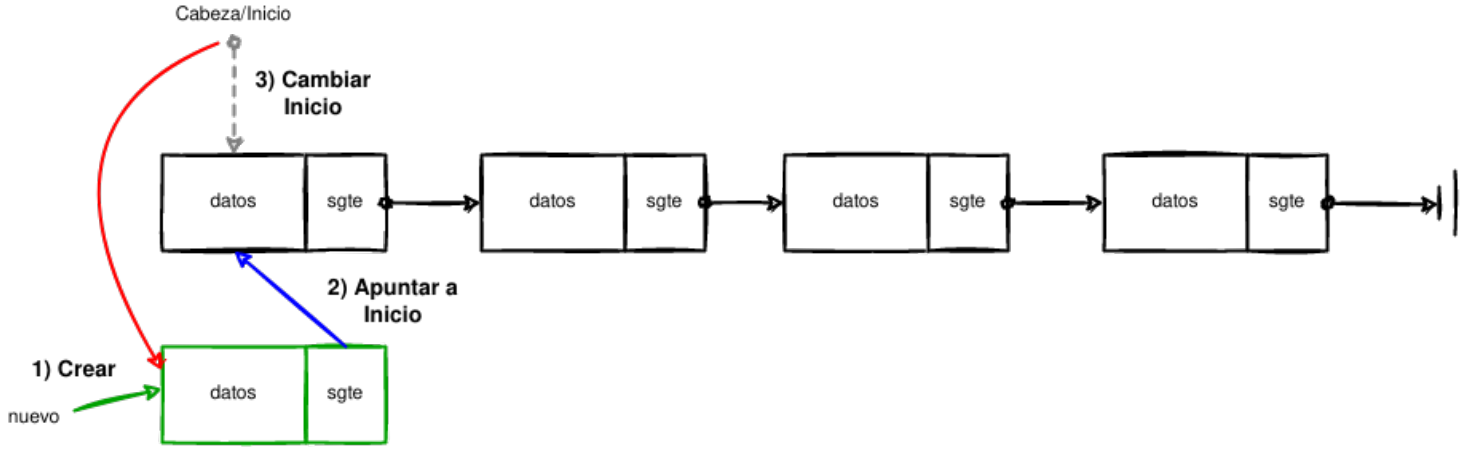
Esquema:



Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Puntero siguiente de nuevo (`nuevo->sgte`) apunta a `inicio`.
4. Puntero `inicio` apunta a nuevo.

Lista Simple Enlazada: Añadir al inicio

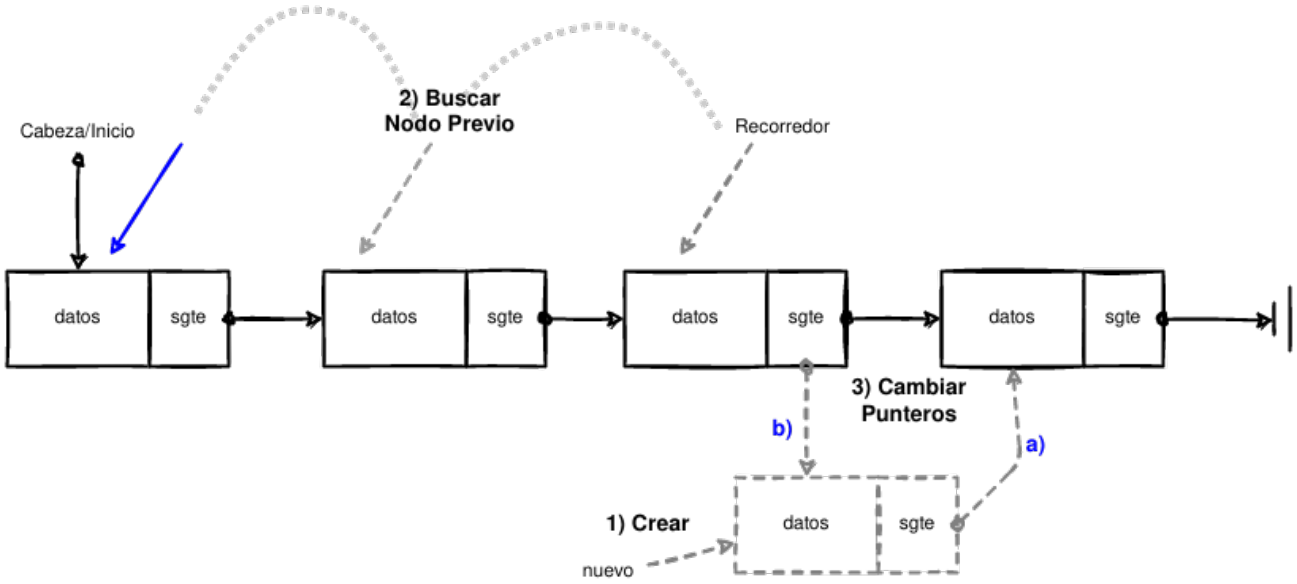


C

```
void add_first(nodo_t **cabeza, int dato){

        nodo_t *nuevo = malloc(sizeof(nodo_t)); // 1) crea nodo
        nuevo->dato = dato; // 2) asigna dato
        nuevo->sgte = *cabeza; // 3) siguiente apunta a inicio
        *cabeza = nuevo; // 4) inicio apunta a nuevo
}
```

Lista Simple Enlazada: Añadir entre nodos

Esquema:

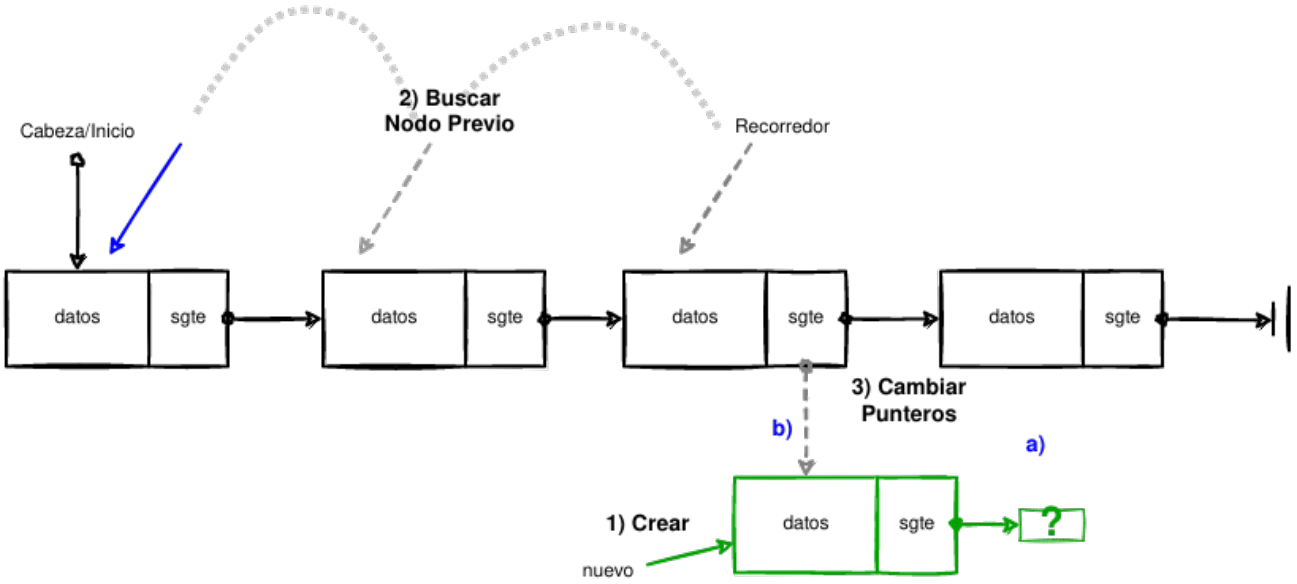


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo previo al objetivo: por valor o posición.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir entre nodos

Esquema:

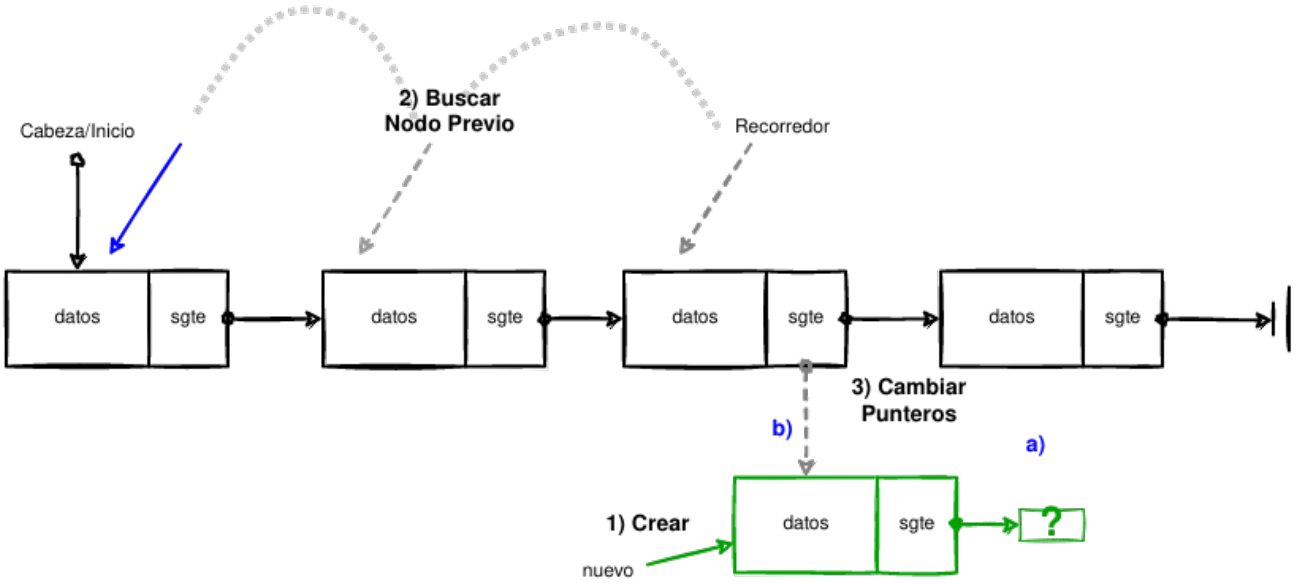


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo previo al objetivo: por valor o posición.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir entre nodos

Esquema:

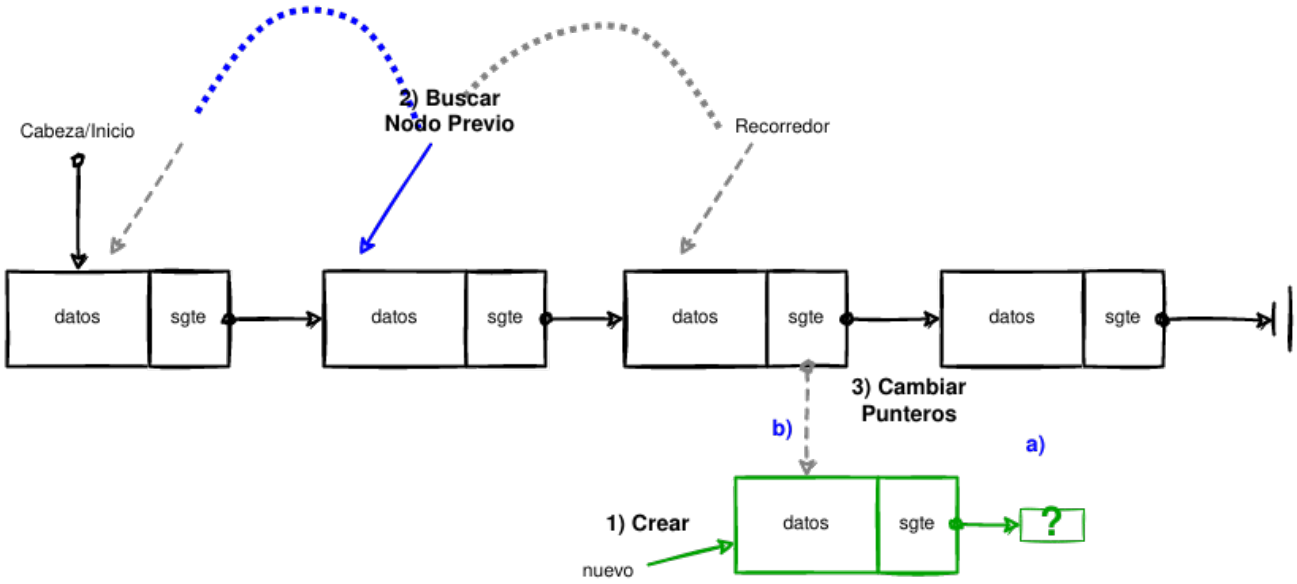


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo previo al objetivo: por valor o posición.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir entre nodos

Esquema:

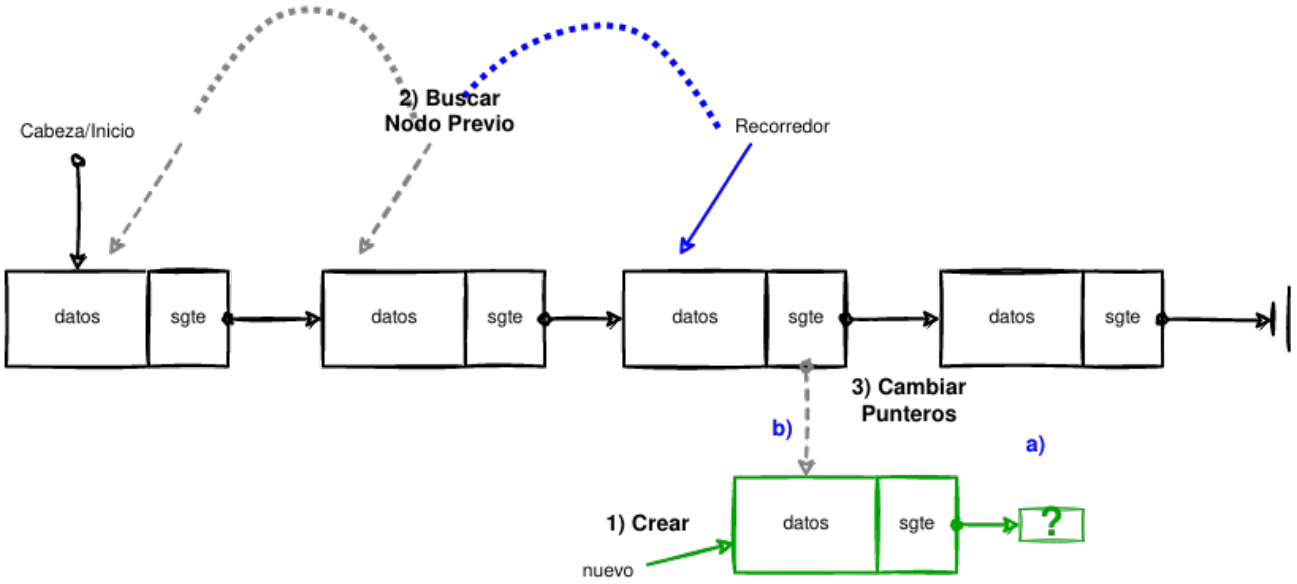


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo previo al objetivo: por valor o posición.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir entre nodos

Esquema:

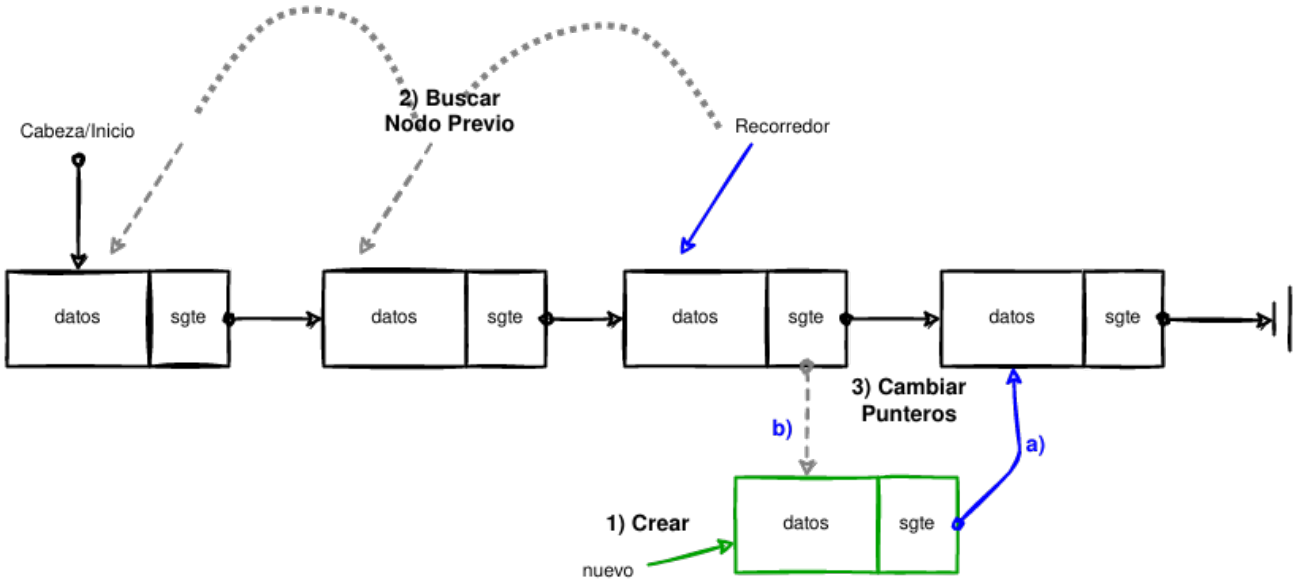


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo previo al objetivo: por valor o posición.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir entre nodos

Esquema:

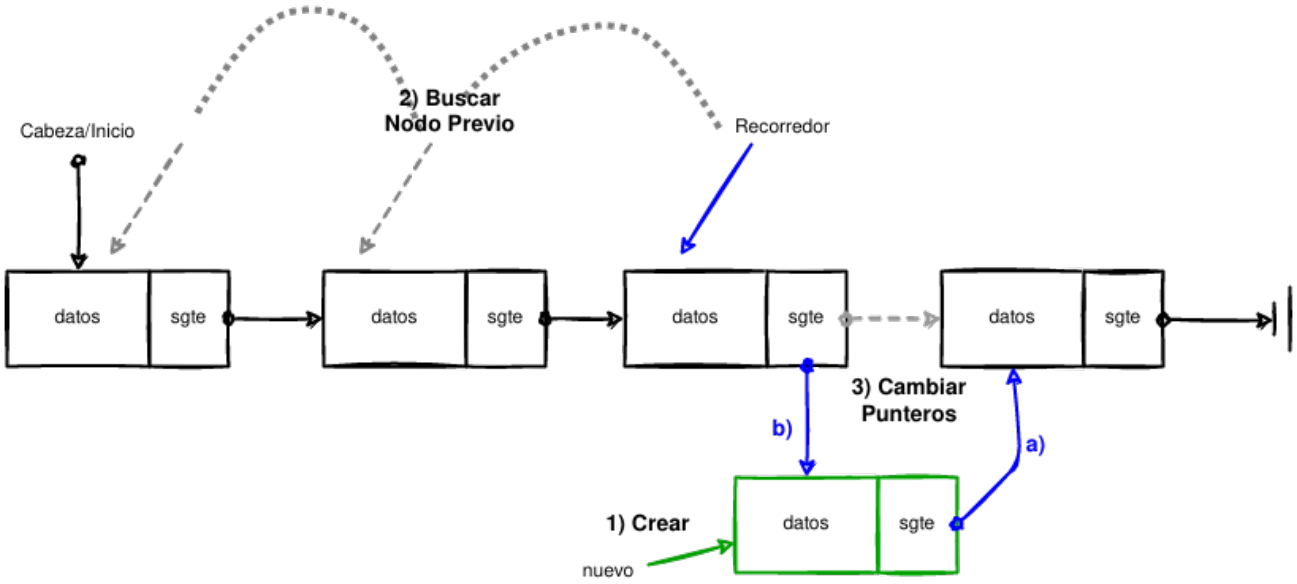


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo previo al objetivo: por valor o posición.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir entre nodos

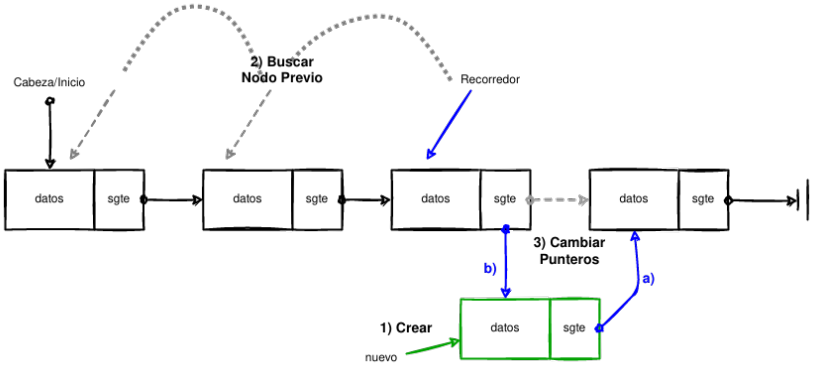
Esquema:



Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo previo al objetivo: por valor o posición.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir entre nodos

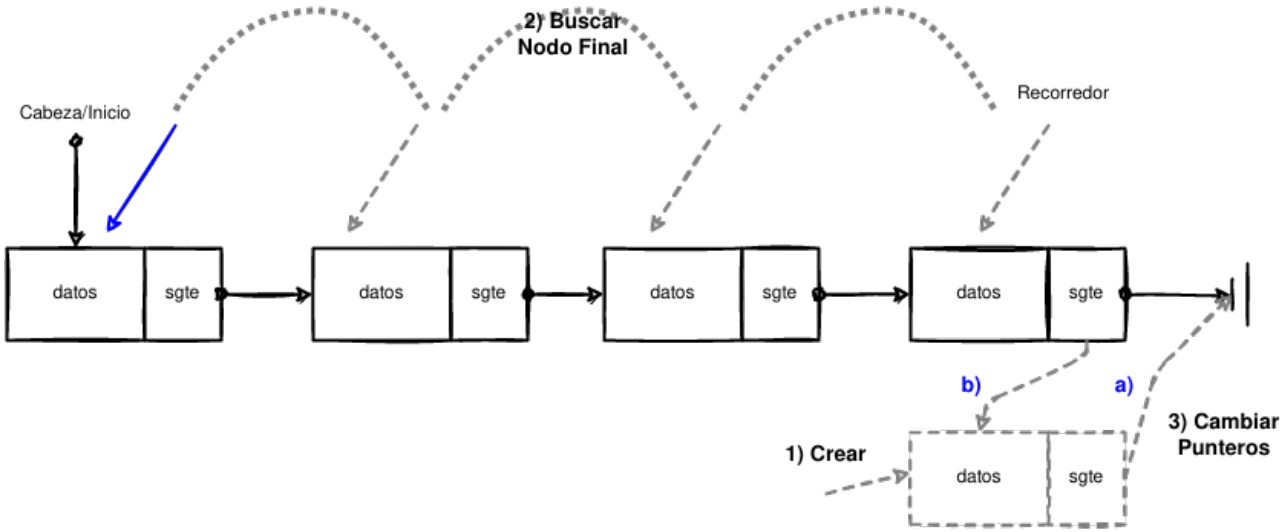


C

```c
// no se modifica el puntero inicio, no es necesario el **
void add_middle(nodo_t *cabeza, int dato, int pos){

        nodo_t *nuevo = malloc(sizeof(nodo_t)); // 1) crea nodo
        nuevo->dato = dato; // 2) asigna dato

        int cont; // nos ayudará a ver la posición en que estamos
        nodo_t *recorredor = cabeza; // ptr que recorre la lista.

        for(cont=0; cont<pos-1; cont ++){ //pos-1, para encontrar el previo
                recorredor = recorredor->sgte; //avanza al siguiente nodo
        } // 3) al finalizar el for, tenemos el nodo previo identificado

        nuevo->sgte = recorredor->sgte; // 4) cambio de punteros
        recorredor->sgte = nuevo; // El orden importa. Memory Leak!
}
```

Lista Simple Enlazada: Añadir al final.

Esquema:

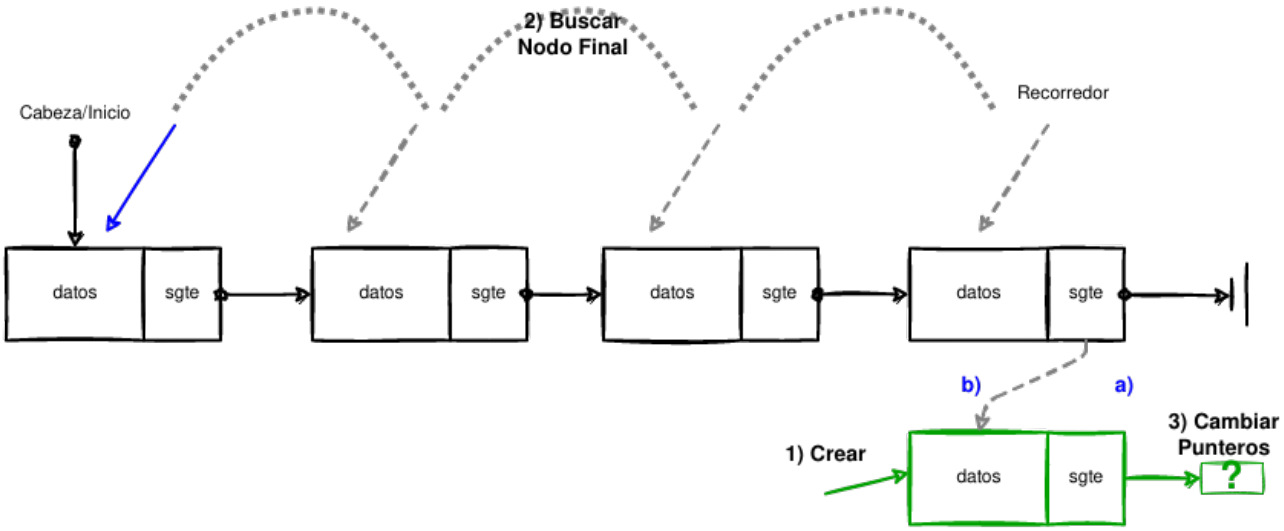


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo final.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir al final.

Esquema:

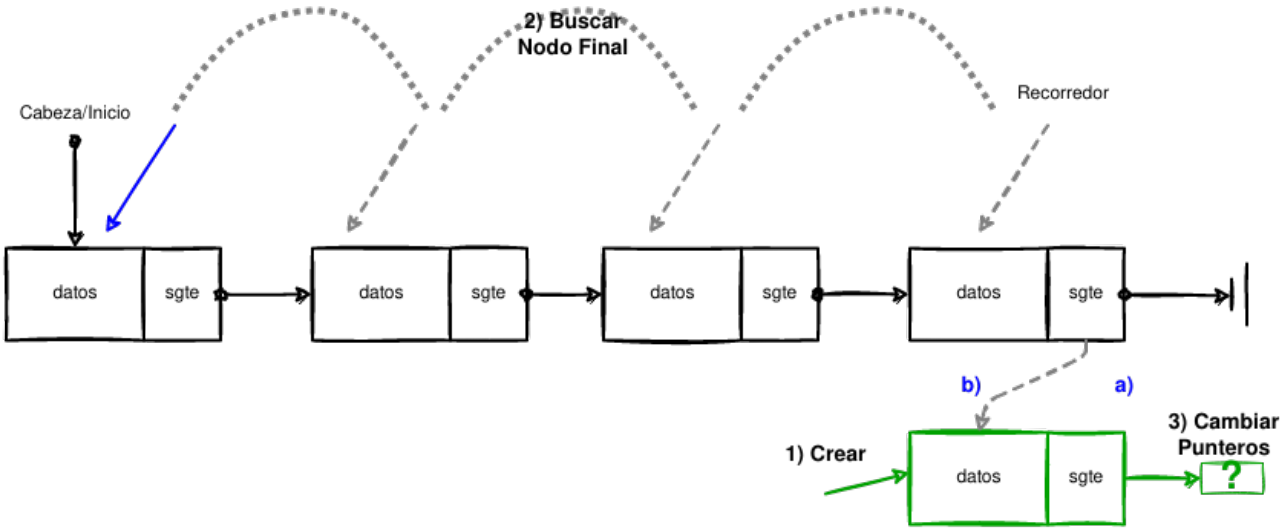


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo final.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir al final.

Esquema:

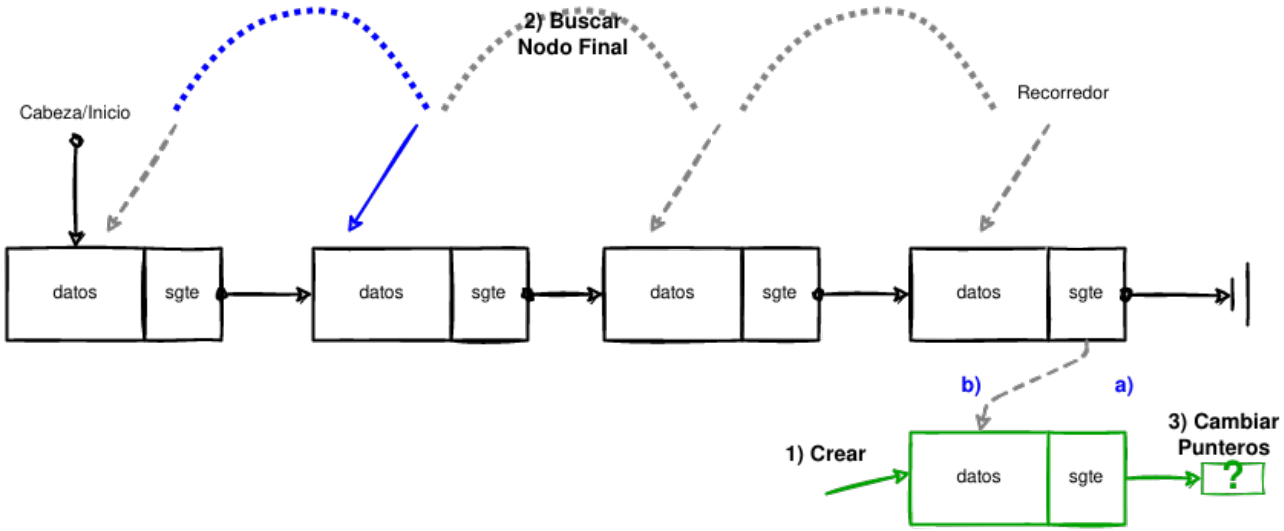


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo final.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir al final.

Esquema:

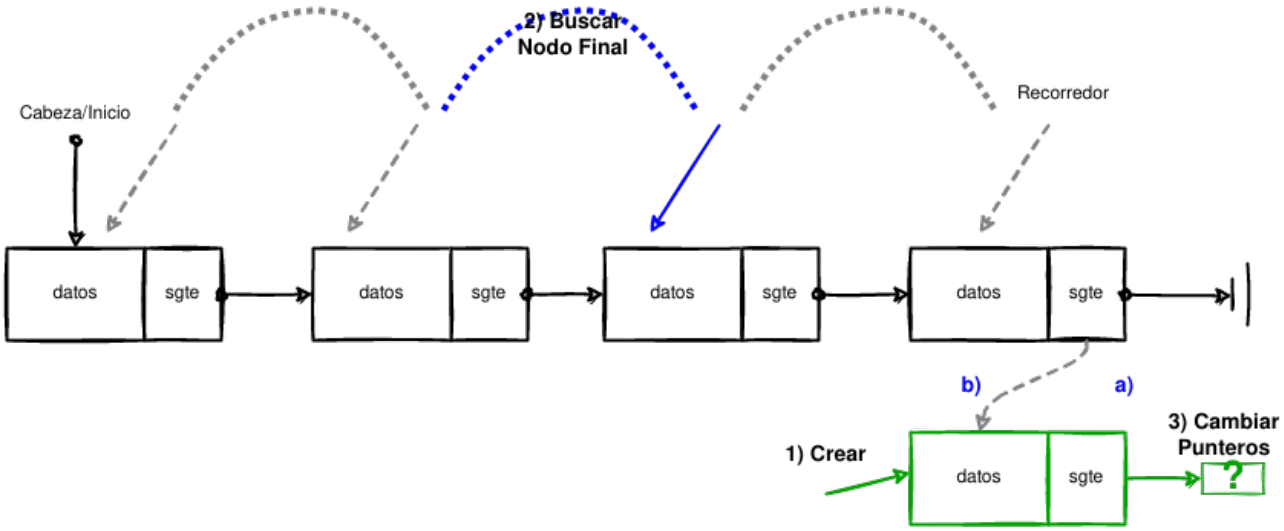


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo final.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir al final.

Esquema:

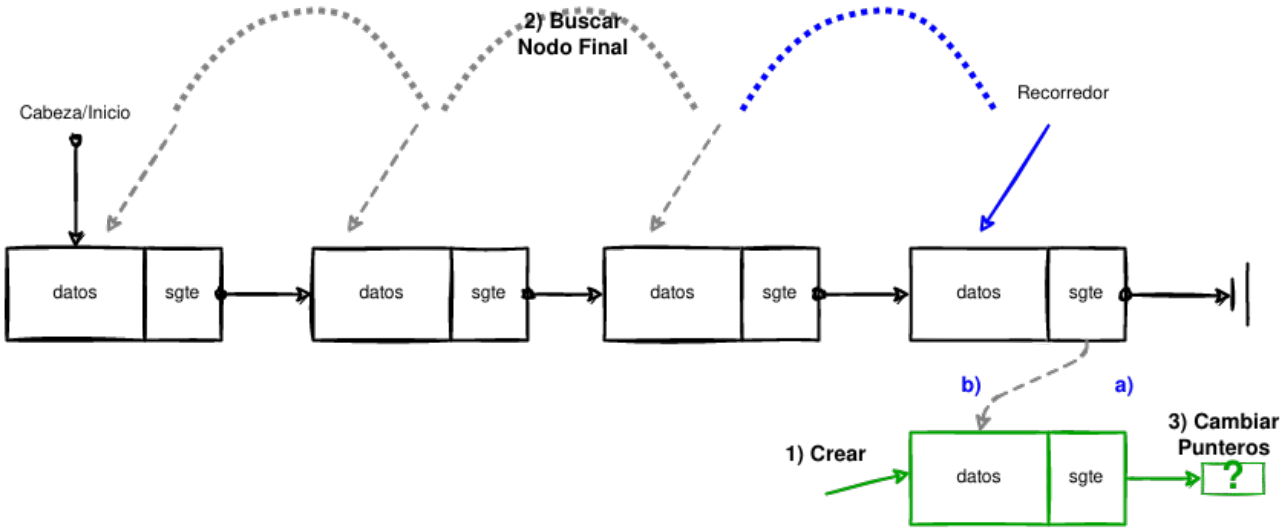


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo final.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir al final.

Esquema:

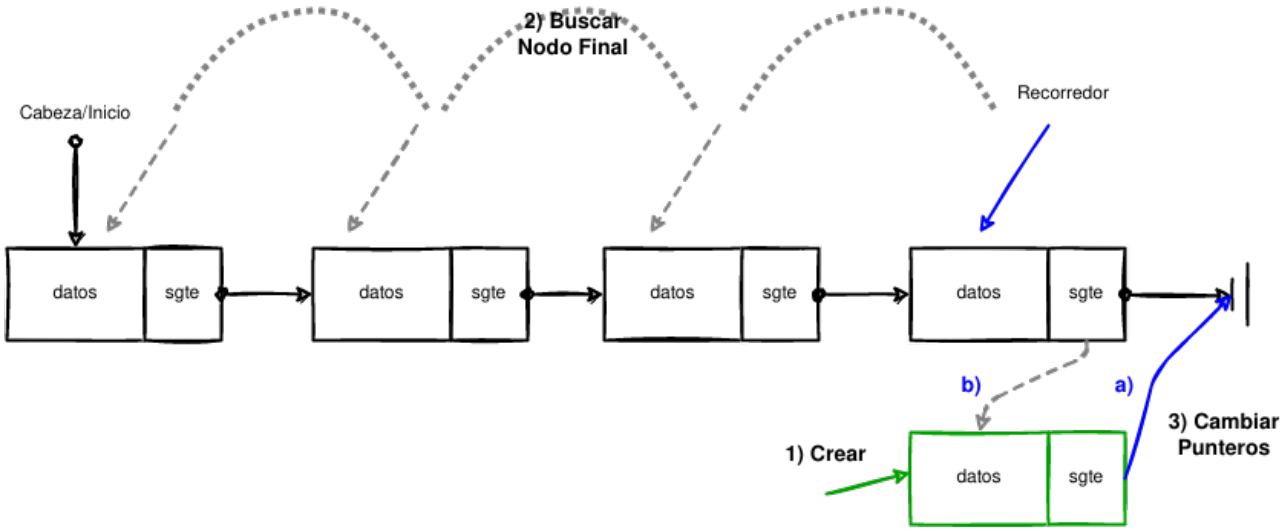


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo final.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir al final.

Esquema:

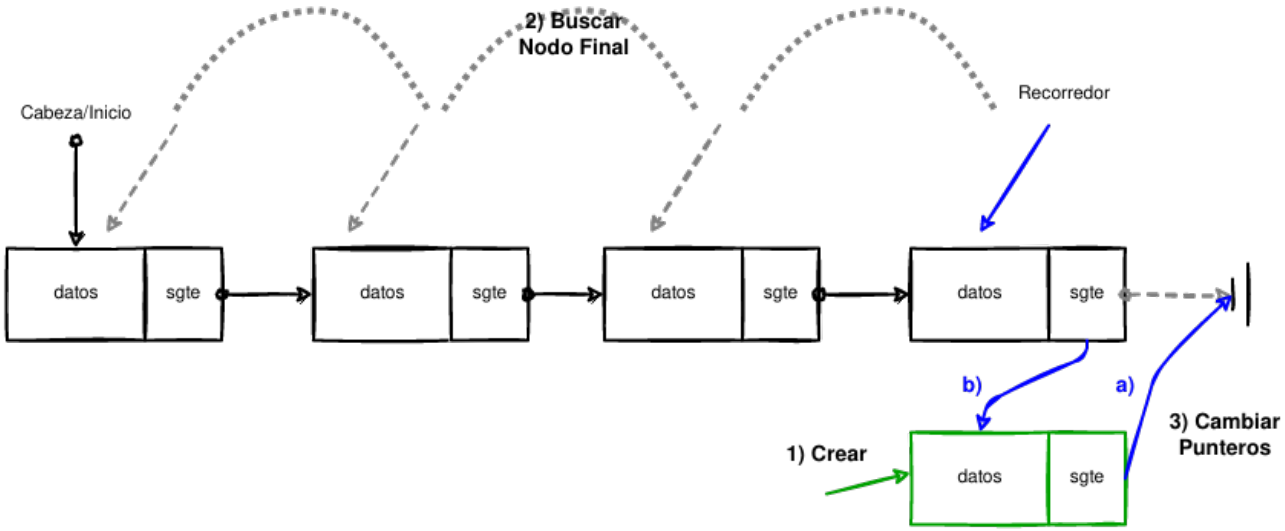


Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo final.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir al final.

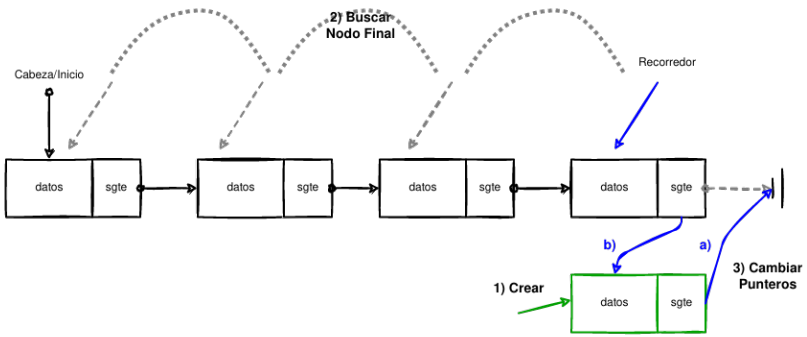
Esquema:



Debemos:

1. Crear el nodo `nuevo`.
2. Asignar el dato.
3. Buscar nodo final.
4. Cambiar punteros.

Lista Simple Enlazada: Añadir al final



C

```
void append(nodo_t *cabeza, int dato){

        nodo_t *nuevo = malloc(sizeof(nodo_t)); // 1) crea nodo
        nuevo->dato = dato; // 2) asigna dato

        nodo_t *recorredor = cabeza; // ptr que recorre la lista.
        while(recorredor->sgte!=NULL){
                recorredor=recorredor->sgte; // 3) avanzamos hasta llegar al final
        }

        nuevo->sgte = recorredor->sgte; // 4) cambio de punteros
        recorredor->sgte = nuevo; // El orden importa. Memory Leak!
}
```

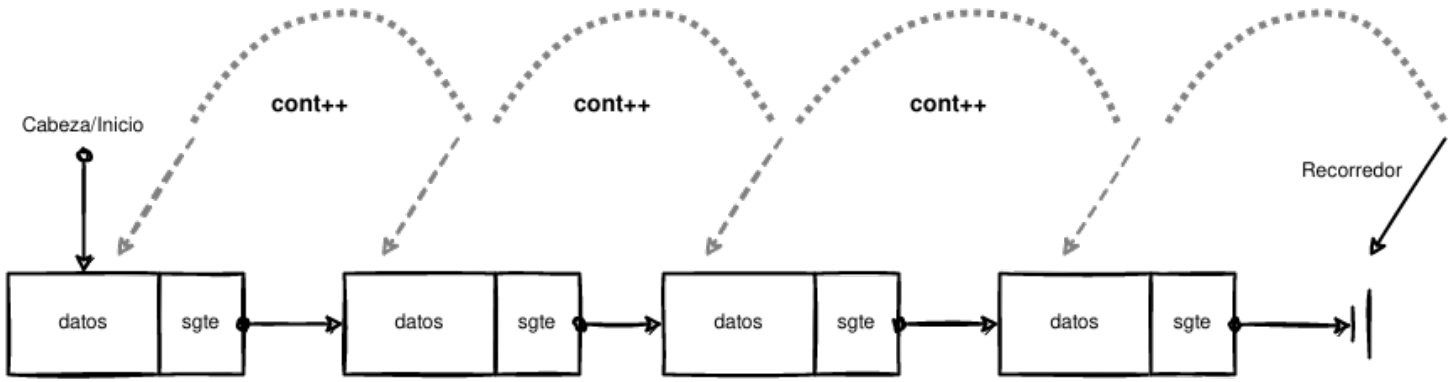
Lista Simple Enlazada: Añadir general

C

```
void add_node(nodo_t **cabeza, int dato, int pos){
        nodo_t * nuevo = malloc(sizeof(nodo_t));
        nodo_t * recorredor = (*cabeza);
        int cont;

        if (nuevo == NULL){
                printf("Error en creación de nodo\n");
                exit(-1);
        }
        nuevo->dato = dato;

        if(recorredor==NULL || pos==0){ //casos primer nodo
                nuevo->sgte=(*cabeza);
                *cabeza = nuevo;
        }else{ //caso entre nodos y final
                for(cont=0; cont<pos-1; cont++){
                        if(recorredor->sgte == NULL) //para nodo final
                                break;
                        recorredor = recorredor->sgte;
                }
                nuevo->sgte = recorredor->sgte;
                recorredor->sgte = nuevo;
        }
}
```

Lista Simple Enlazada: Largo de lista.

Seguramente, considerarían más sencillas las funciones anteriores si tuvieremos el largo de la lista. Siguiendo el principio de recorrer:



C

```
int list_len(nodo_t *cabeza){

        nodo_t *recorredor = cabeza;
        int cont;

        for(cont=0; recorredor!=NULL; cont++){
                recorredor = recorredor -> sgte;
        }

        return cont;
}
```

Lista Simple Enlazada: Eliminar nodos

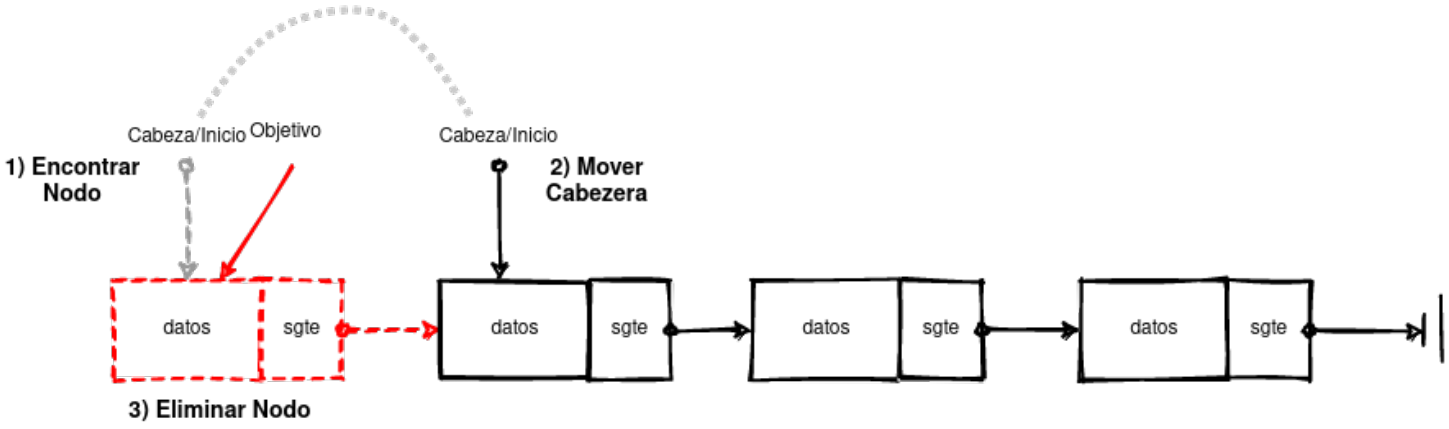
Tendremos 2 casos a tener en consideración. Uds, deberán implementarlos a partir de las figuras.

- Eliminar primer nodo.
- Eliminar nodo intermedio o final.

Ojo, que no siempre se pueden borrar nodos: **lista vacía**.

Lista Simple Enlazada: Eliminar primero nodo

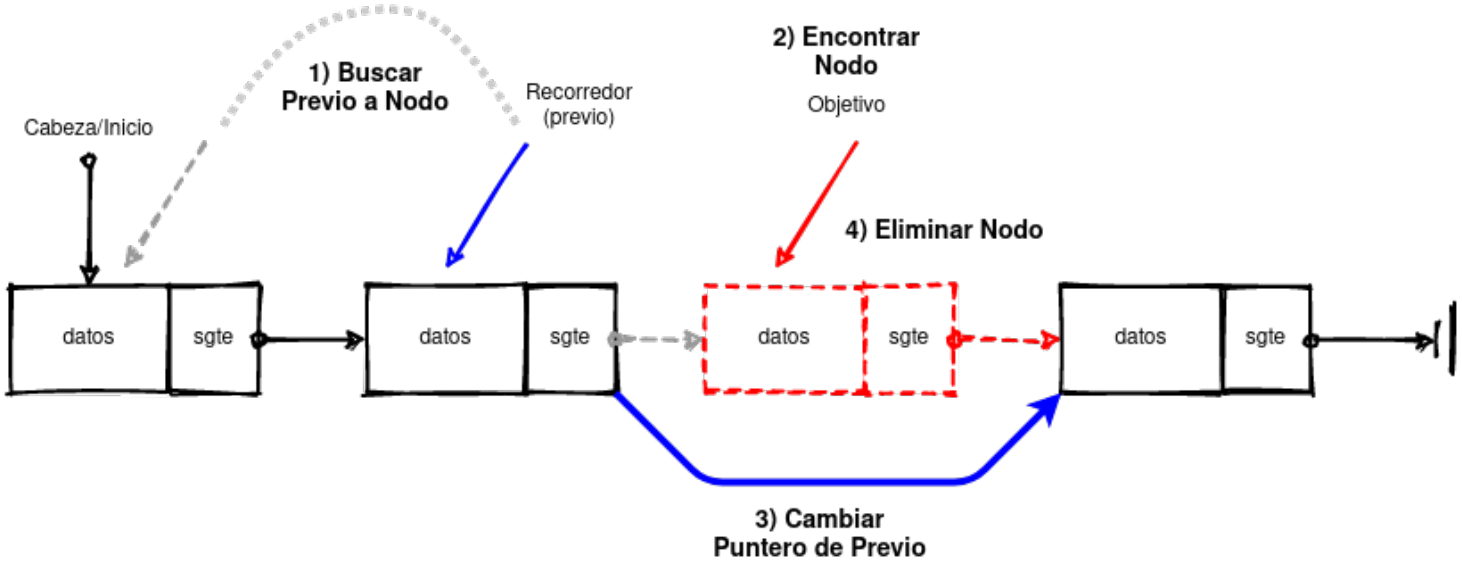
Implemente la siguiente figura:



Ojo, que no siempre se pueden borrar nodos: **lista vacía**.

Lista Simple Enlazada: Eliminar nodo intermedio o final

Implemente la siguiente figura:



Ojo, que no siempre se pueden borrar nodos: **lista vacía**.

Lista Simple Enlazada: Imprimir Lista.

Para verificar que la lista está en cierto estado, pueden crear la función imprimir lista. Explique como funciona con una figura.

C

```c
void print_list(nodo_t *cabeza){

        // rec = recorredor ;)
        nodo_t *rec = cabeza;
        int cont;

        printf("--- Print Lista ---\n");
        for(cont=0; rec!=NULL; cont++){
                printf("Lista pos %d, valor: %d\n", cont, rec->dato);
                rec = rec -> sgte;
        }
        printf("--- Fin Lista ---\n\n");

}
```

Lista Simple Enlazada: Limpiar Lista.

Recuerden que al final de cada uso, toda la memoria reservada **debe ser liberada**. Por lo tanto, esta función es obligatoria. Explique como funciona con una figura.

C

```c
void free_list(nodo_t *cabeza){

        // rec = recorredor ;)
        nodo_t *rec = cabeza;

        while(rec!=NULL){
                cabeza = rec->sgte;
                free(rec);
                rec=cabeza;
        }
        //¿qué le pasa al puntero cabecera?
        //¿cómo lo arreglan?
}
```

Dangling

- **Dejar Colgado (dangling).**
- Puntero hacia localización de memoria del heap que ha sido liberada (e incluso nuevamente asignada).
- Genera errores poco predecibles (e.g. Segmentation fault, corrupción de memoria).

C

```
int main(int argc, char **argv){
        int x=2;
        int *p, *q;

        p= malloc(sizeof(int));
        q=p;

        free(p);  /* q aun mantiene referencia al heap */
                 /* puede que variable de heap sea reasignada */

        *q = x;

        return 0;
}
```

Fuga de memoria - Memory Leak

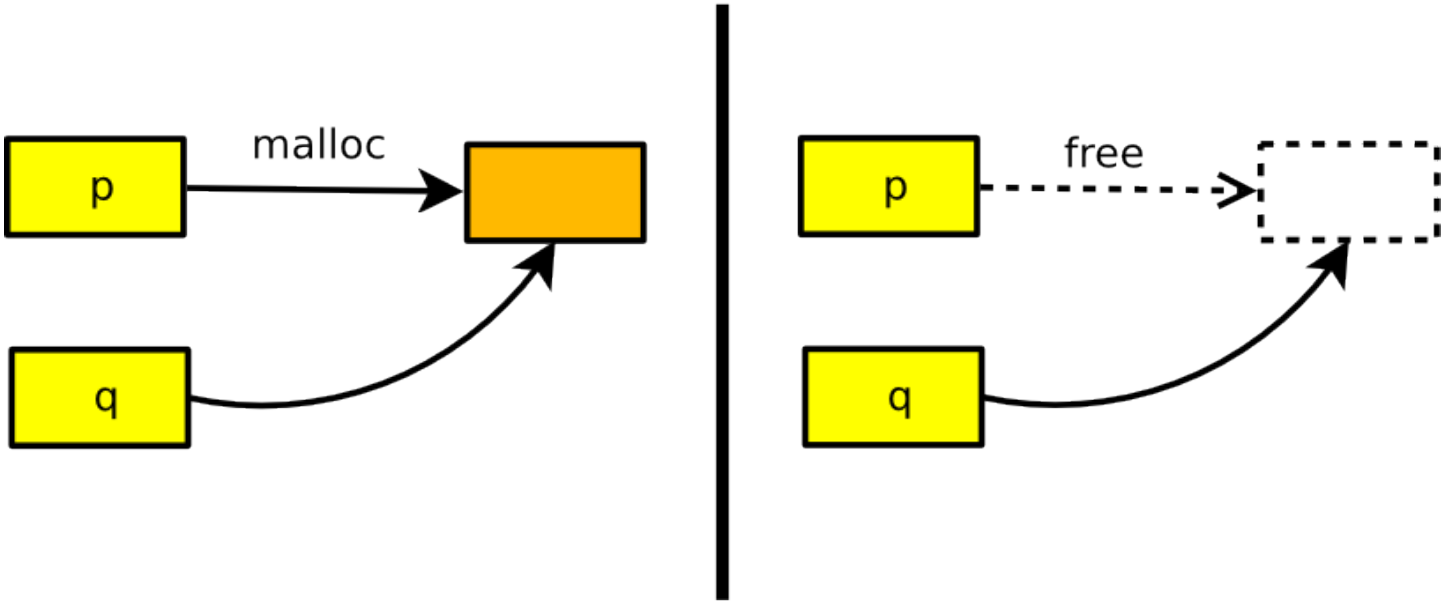
- **Fuga de memoria (memory leak)**.
- Pérdida de acceso a objeto de memoria reservado en el heap, por no existir variables que apunten a él.
- No se puede liberar hasta el término del programa.
- Puede saturar la memoria.

C

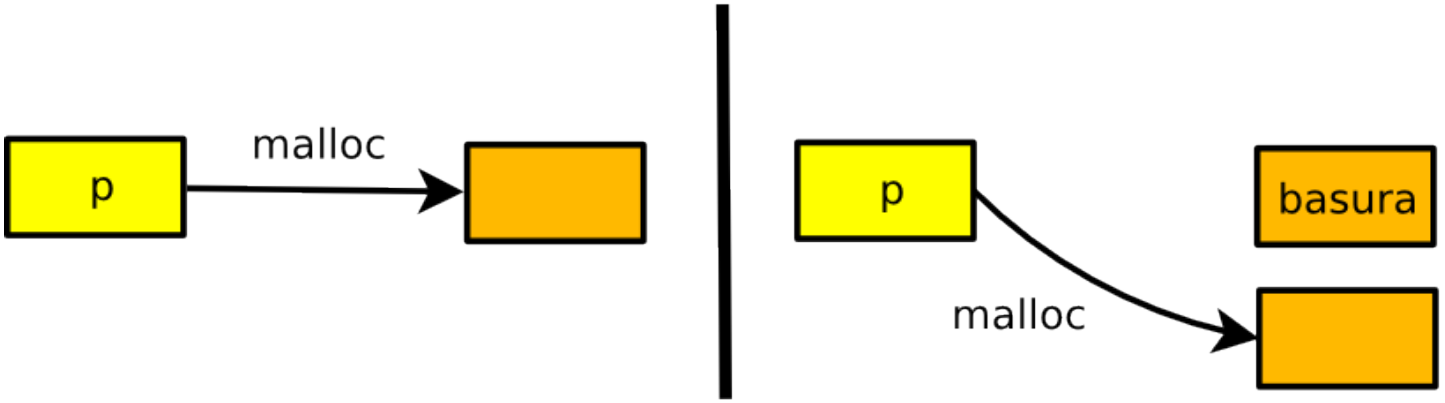
```
int main(int argc, char **argv){
        int *p;

        p= malloc(sizeof(int));
        p= malloc(sizeof(int));
        /* se pierde la direccion asignada anteriormente */

        free(p);

        return 0;
```

Ejemplo

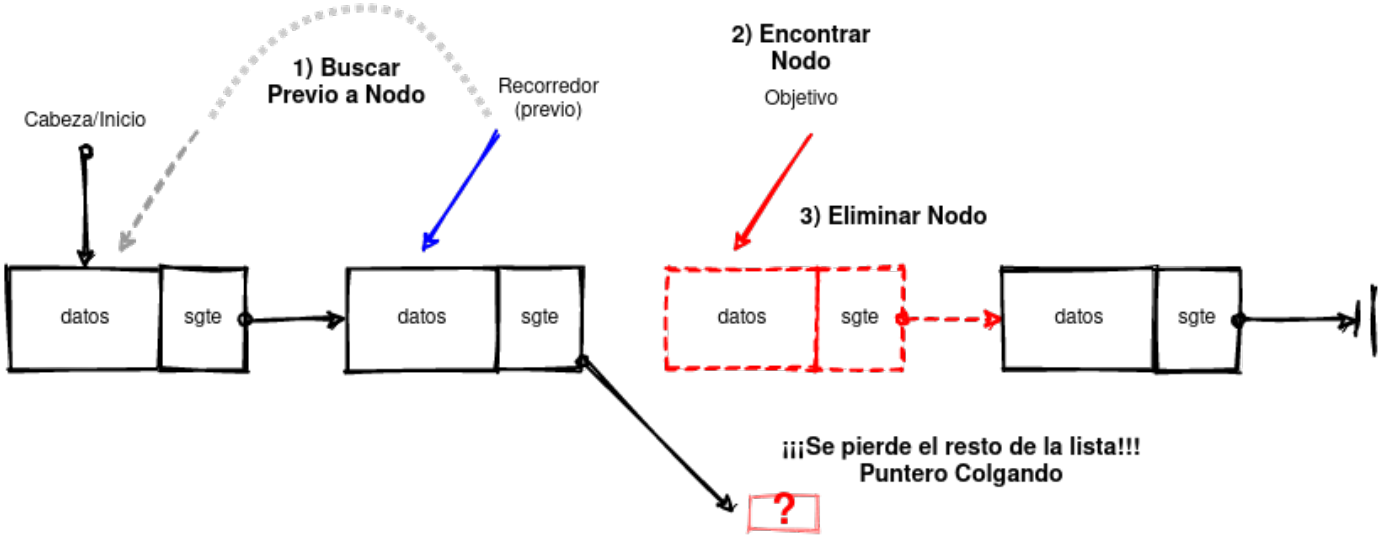
Dangling



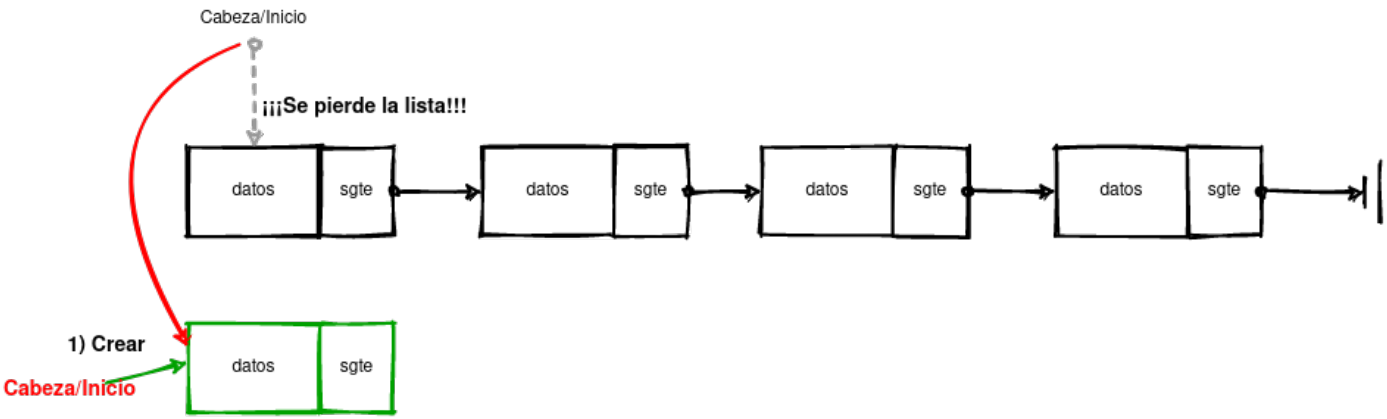
Memory Leak



Dangling



Memory Leak



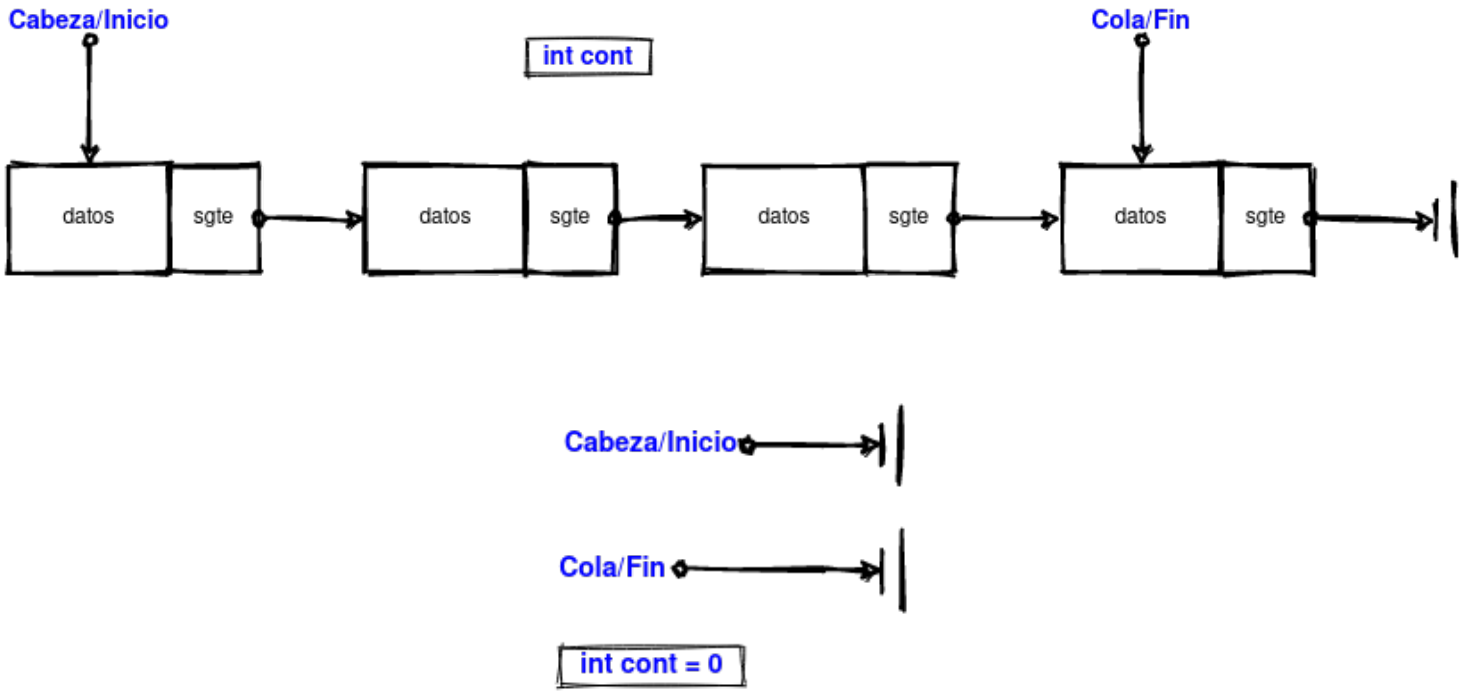
Lista Enlazada PRO

Existen otras implementaciones de listas enlazadas simples, donde se incluyen más punteros y variables de control.

C

```c
#include<stdio.h>

typedef struct nodo{
        int dato; //dato, puede ser lo que uds. necesiten
        struct nodo *sgte; // puntero a un nodo enalce.
} nodo_t;

typedef struct list{
        nodo_t * inicio;
        nodo_t * final;
        int largo;
} lista;

int main(int argc, char **argv){
        lista *L = malloc(sizeof(lista));
        L->inicio = NULL;
        L->final = NULL;
        L->largo = 0;
        // Equivalente a lista vacía []
        // ¿Para que sirve esta nueva forma?
        // Implemente las funciones de ésta clase considerando esta forma.

        free(L);
        return 0;
}
```

Lista Enlazada PRO



Lista de Python en C

Implemente una lista, para algún tipo de dato a su elección, que incluya las funciones más conocidas de las listas de Python:

- `list()` o [].
- `.insert(pos, value)`
- `.append(value)`
- `.clear()`
- `del lista[pos]`
- `.remove(value)`
- `.sort()`
- `.reverse()`
- `.count(value)`
- `.index(value)`